

Token Provider

الجزء المسؤول عن جيبان ال token اللي بنكلم بيه CRM

1. الفكرة

أي request رايح ل CRM لازم يكون معاه token. ال token ده بتاع ال **API نفسها**، مش بتاع اليوزر، وكل اليوزرز بيستخدموا نفس ال token.

قسمنا الشغل لحايتين:

- **TokenProvider** : **العقد**، يعني "أي حد بيدّيني token".
- **MsalTokenProvider** : **التنفيذ**، اللي بيحجب ال token فعلاً من Azure باستخدام مكتبة MSAL.

2. TokenProvider: له abstract class مش interface؟

```
export abstract class TokenProvider {  
  abstract getToken(forceRefresh?: boolean): Promise<string>;  
}
```

ال **interface** في TypeScript موجود بس وقت كتابة الكود. بعد ال compile ل JavaScript بيختفي **خالص**، ف NestJS مش بيلاقى حاجة يستخدمها في ال Dependency Injection.

ال **abstract class** بيفضل موجود بعد ال compile، فنقدر نستخدمه ك key في ال module:

```
{ provide: TokenProvider, useClass: MsalTokenProvider }  
// whoever asks for TokenProvider gets MsalTokenProvider
```

الفايدة:

- أي **class** بيستخدم ال **token** **بيعتمد على العقد بس**، ومش عارف إن فيه MSAL أصلاً (Dependency Inversion).
- **سهل نغيره**: لو احتجنا طريقة تانية نجيب بيها ال token، هنعمل class جديد ونغير سطر واحد في ال module.
- **سهل نختبره**: في ال tests بنحط class fake بيرجع 'fake-token'، من غير ما نكلم Azure.

3. MsalTokenProvider ال fields وال constructor

```
private readonly msal: ConfidentialClientApplication;
private readonly scope: string;

constructor(config: ConfigService) { ... }
```

ليه `msal` و `scope` متخزين كـ `fields`؟ لأننا محتاجينهم كل مرة نطلب فيها `token`، يعني جوه `getToken`.

ليه `config` مش متخزن؟ لأننا محتاجينه مرة واحدة بس في ال `constructor` عشان نقرا القيم من ال `env`. بعد كده مالوش لازمة، فمش بنحتفظ بيه. كده ال `class` محتفظ بس باللي هو محتاجه فعلاً.

ملاحظة: لو كتبنا `private readonly config: ConfigService` جوه ال `TypeScript`، `constructor` كان هيحفظه field لوحده.

الكلمة	معناها
<code>private</code>	محدث برا ال <code>class</code> يقدر يوصله.
<code>readonly</code>	بيتحط مرة واحدة في ال <code>constructor</code> ومينفعش يتغير بعد كده.

4. this.msal و this.scope

```
this.msal = new ConfidentialClientApplication({
  auth: {
    clientId: DV_CLIENT_ID, // who we are
    clientSecret: DV_CLIENT_SECRET, // our password
    authority: login.microsoftonline.com/DV_TENANT_ID, // who issues the token
  },
});
this.scope = `${DV_URL}/.default`;
```

`this.msal`: ال `object` اللي بيكلم Azure بالنيابة عنا. فيه حاجتين مهمين:

- عارف إحنا مين (`clientId + clientSecret`)، ونطلب ال `token` من أنهي (`authority`) **Azure tenant**.
- جواه ال `cache`. أول `token` بيتحفظ في ال `memory`، والطلبات اللي بعده بتاخد نفس ال `token` لحد ما يقرب يخلص (حوالي ساعة).

عشان كده `msal` لازم يتعمل مرة واحدة بس في ال `constructor`. لو عملناه جوه `getToken`، كل `request` هيبدا بـ `cache` فاضي ويطلب `token` جديد من Azure.

`this.scope`: ال `token` ده هنستخدمه فين. `DV_URL/.default` معناها: "عاوز `token` لـ Dataverse ده، بكل الصلاحيات اللي اتدّت لـ `app`".

```

async getToken(forceRefresh?: boolean): Promise<string> {
  const result = await this.msal.acquireTokenByClientCredential({
    scopes: [this.scope],
    skipCache: forceRefresh,
  });

  if (!result?.accessToken) {
    throw new Error('Azure AD returned no access token');
  }

  return result.accessToken;
}

```

اللي بيحصل	forceRefresh
الحالة العادية. MSAL بيرجّع ال token من ال cache لو لسه صالح، ولو مفيش بيحيب واحد جديد.	مش متبعت / false
MSAL بيتجاهل ال cache وبيحيب token جديد من Azure على طول.	true

هنستخدم `forceRefresh = true` إمتى؟ لما CRM يرجّع **401**، يعني رفض ال token. ساعتها `DataVerseClient` هيطلب token جديد ويجزّب ال request مرة واحدة بس.

النتيجة:

- مفيش `result` (`token` أو `accessToken` فاضي): بنعمل `throw Error`. مينفعش نكمّل ونبعت request ل CRM من غير token.
- ال token موجود: بنرجّعه.

الخلاصة

- `TokenProvider` هو العقد، وكان لازم يبقى `abstract class` عشان NestJS يقدر يستخدمه في ال DI.
- `MsalTokenProvider` بيقرأ ال config مرة واحدة، وبيحتفظ بـ `msal` و `scope` بس.
- `msal` بيتعمل مرة واحدة عشان ال cache يفضل موجود.
- `getToken()` بياخد ال token من ال cache، و `getToken(true)` بيحيب واحد جديد من Azure. ولو مفيش token بيعمل `Error`.